

MANUAL TÉCNICO

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

Facultad de Ingeniería

Nombre: Jonathan Eduardo Fuentes Garcia

Carnet: 202408977

Práctica 1 – Segundo Semestre 2026



1. Introducción

El presente manual técnico describe la arquitectura, diseño e implementación del sistema desarrollado para la práctica. El sistema consiste en un analizador léxico y sintáctico de comandos EXT2, que recibe comandos desde una interfaz web, los valida estructuralmente y retorna mensajes de éxito o error, sin ejecutar ninguna acción real sobre el sistema de archivos.

2. Arquitectura del Sistema

El sistema está compuesto por dos componentes principales:

- **Backend:** desarrollado en C++ (estándar C++17). Se encarga del análisis léxico y sintáctico de los comandos. Expone una API REST mediante un servidor HTTP implementado con la biblioteca `cpp-http-lib`.
- **Frontend:** desarrollado con React (JavaScript). Proporciona una interfaz gráfica para que el usuario ingrese comandos, visualice resultados y mantenga un historial de ejecución.

La comunicación entre ambos se realiza mediante peticiones HTTP POST en texto plano.

3. Estructuras de Datos Utilizadas

3.1. CommandResult (definida en Commands/CommandResult.h)

```
struct CommandResult {  
    bool success; // true si el comando es válido  
    std::string message; // mensaje de éxito o error  
};
```

Esta estructura es retornada por cada función validadora de comando.

4. Análisis Léxico y Sintáctico

4.1. Tokenización

El backend recibe una línea de texto que contiene el comando y sus parámetros. La función `campos()` (en `Analyzer/Analyzer.cpp`) divide la línea en tokens, respetando las comillas dobles para que las rutas con espacios se mantengan como un solo token.

```
static std::vector<std::string> campos(const std::string& linea) {  
    // Recorre carácter por carácter, respetando comillas  
}
```

4.2. Reconocimiento de Comandos

El primer token se identifica como el nombre del comando (en minúsculas). Luego se delega la validación a la función específica de cada comando (por ejemplo, `Mkdisk_Command`).

4.3. Validación de Parámetros

Cada comando utiliza expresiones regulares de la biblioteca `<regex>` para validar la estructura de los parámetros y sus valores. Por ejemplo, para `MKDISK` se usa:

```
static const std::regex expresion(
    R"(-size=\d+|-unit=[km]|-fit=[b|f|w]{2})|-path="[^"]+"|-path=[^\s]+)",
    std::regex::icase
);
```

Cada parámetro se extrae, se valida su categoría si es obligatorio u opcional sus valores permitidos y restricciones (como longitud máxima de 10 caracteres en MKUSR).

5. Comandos Implementados

Los comandos implementados son los siguientes:

Comando	Archivo de implementación	Parámetros obligatorios	Parámetros opcionales
MKDISK	Commands/Mkdisk.cpp	-size, -path	-unit, -fit
RMDISK	Commands/Rmdisk.cpp	-path	-
FDISK	Commands/Fdisk.cpp	-size, -path, -name	-unit, -type, -fit
MOUNT	Commands/Mount.cpp	-path, -name	-
MKFS	Commands/Mkfs.cpp	-id	-type
MKUSR	Commands/Mkusr.cpp	-user, -pass, -grp	- (todos obligatorios)
RMUSR	Commands/Rmusr.cpp	-user	-

MKFILE	Commands/Mkfile.cpp	-path	-f, -size, -cont
--------	---------------------	-------	------------------

Cada función validadora retorna un CommandResult con un mensaje descriptivo.

6. Comunicación con el Frontend (API REST)

El servidor HTTP (en main.cpp) escucha en <http://localhost:8080> y expone el endpoint POST /analyze. El cuerpo de la petición contiene el comando a analizar. La respuesta es el mensaje generado por el analizador .

Ejemplo de petición con curl:

```
curl -X POST http://localhost:8080/analyze -d "mkdisk -size=10 -
path=/tmp/prueba.mia"
```

Respuesta esperada:

[OK] MKDISK ejecutado corectamente. Tamaño: 10M, Ajuste: FF, Rua: /tmp/prueba.mia

7. Instrucciones de Compilación y Ejecución

7.1. Requisitos previos

- Sistema operaivo Linux (instalación física, no VM, según requisito).
- Compilador GCC con soporte C++17.
- make, g++, pthread.
- Node.js y npm para el frontend.

7.2. Backend

```
cd backend
```

```
make clean
```

```
make run
```

El servidor se iniciará en el puerto 8080.

7.3. Frontend

cd frontend

npm install

npm start

La aplicación estará disponible en <http://localhost:3000>.

8. Dependencias Externas

- **cpp-httplib** (versión 0.52.0): incluida como archivo httplib.h en el directorio backend/.
- **React** y **axios**: instalados mediante npm.

9. Conclusiones

Se ha implementado un analizador completo que cubre todos los comandos solicitados. La arquitectura separa correctamente la lógica de análisis (backend C++) de la interfaz de usuario (frontend React), cumpliendo con los requisitos técnicos.